ELL787: Embedded Systems and Applications

Assignment Report - II

National Time Network-Synchronised Speaking Clock using STM32, FreeRTOS and QEMU

Submitted by

2024EEN2016 Humanshu D G

Under the supervision of:

Prof. (Dr.) Kaushik Saha

Session 2025 - 2026

Department of Electrical Engineering

Indian Institute of Technology Delhi

Hauz Khas, New Delhi, Delhi, India - 110016

Table of Contents

1 System Architecture **1**
- 1.1 Hardware Platform . . . 1
- 1.2 Software Stack . . . 1
- 1.3 Data Flow . . . 1
- 1.4 Project Directory Structure . . . 2
- 1.5 Memory Map . . . 3

2 RTOS Design **3**
- 2.1 FreeRTOS Configuration . . . 3
- 2.2 Task Architecture . . . 3
- 2.3 Key Task (`src/key_task.c`) . . . 4
- 2.4 NTP Task (`src/ntp_task.c`) . . . 4
- 2.5 Speech Task (`src/speech_task.c`) . . . 4
- 2.6 Inter-Task Communication (IPC) . . . 4
- 2.7 Fault Handlers . . . 5

3 Networking Implementation **5**
- 3.1 lwIP Configuration . . . 5
- 3.2 Network Initialization Sequence . . . 5
- 3.3 LAN9118 Ethernet Driver (`network/netif_qemu.c`) . . . 6
- 3.4 Register Access . . . 6
- 3.5 Initialization (`lan9118_hw_init`) . . . 6
- 3.6 Transmit (`lan9118_low_level_output`) . . . 6
- 3.7 Receive (`lan9118_poll`) . . . 6
- 3.8 QEMU Networking Modes . . . 6
- 3.9 DHCP Flow . . . 6

4 NTP Protocol Explanation **7**
- 4.1 Protocol Overview . . . 7
- 4.2 NTP Packet Structure . . . 7
- 4.3 Client Request Construction . . . 8
- 4.4 Response Parsing . . . 8
- 4.5 Time Conversion . . . 8
- 4.6 NTP Server . . . 8
- 4.7 Speech Token Protocol . . . 9

5 System Testing **9**
- 5.1 Build Verification . . . 9
- 5.2 Boot Sequence Verification . . . 9
- 5.3 NTP Query Test . . . 10
- 5.4 Verification Checklist . . . 10
- 5.5 GDB Debugging . . . 10

6 Instructions to Run **11**
- 6.1 Prerequisites . . . 11
- 6.2 Build . . . 11
- 6.3 Run (Console Only) . . . 11
- 6.4 Run with Speech (TTS Bridge) . . . 11
- 6.5 Makefile Targets Summary . . . 11

6.6 Troubleshooting 11

Problem Statement

A real-time speaking clock running on an emulated **STM32F4** using **QEMU**, **FreeRTOS**, and the **lwIP** TCP/IP stack. When the user presses a key, the system contacts a public NTP server over the network, retrieves the current UTC time, converts it to **Indian Standard Time (IST, UTC+5:30)**, and announces it through a speech token interface that drives a host-side text-to-speech engine.

1 System Architecture

1.1 Hardware Platform

The project targets the **STM32** microcontroller board, emulated entirely in QEMU. This board features:

Component	Detail
CPU	ARM Cortex-M4 (ARMv7-M, Thumb-2 ISA)
System Clock	25 MHz
Flash (ZBT SSRAM1)	4 MB at `0x00000000`
RAM (ZBT SSRAM2/3)	4 MB at `0x20000000`
UART0 (CMSDK)	`0x40004000` — console I/O
Ethernet (LAN9118)	`0x40200000` — built-in to the board model

No physical hardware is required. QEMU's user-mode networking (SLIRP) provides NAT-based Internet access for the emulated Ethernet controller.

1.2 Software Stack

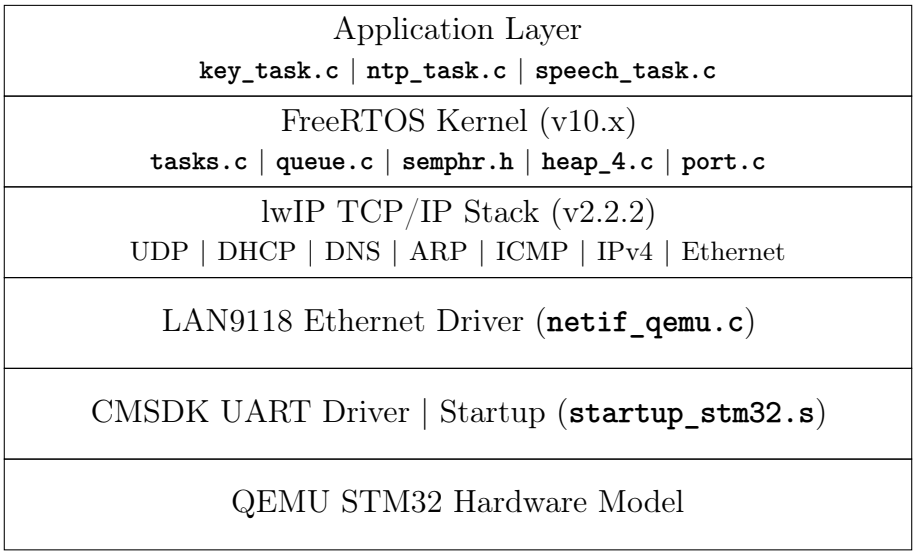

Figure 1.1: Software Stack Architecture

1.3 Data Flow

The system follows a **producer-consumer pipeline** with three FreeRTOS tasks communicating through IPC primitives:

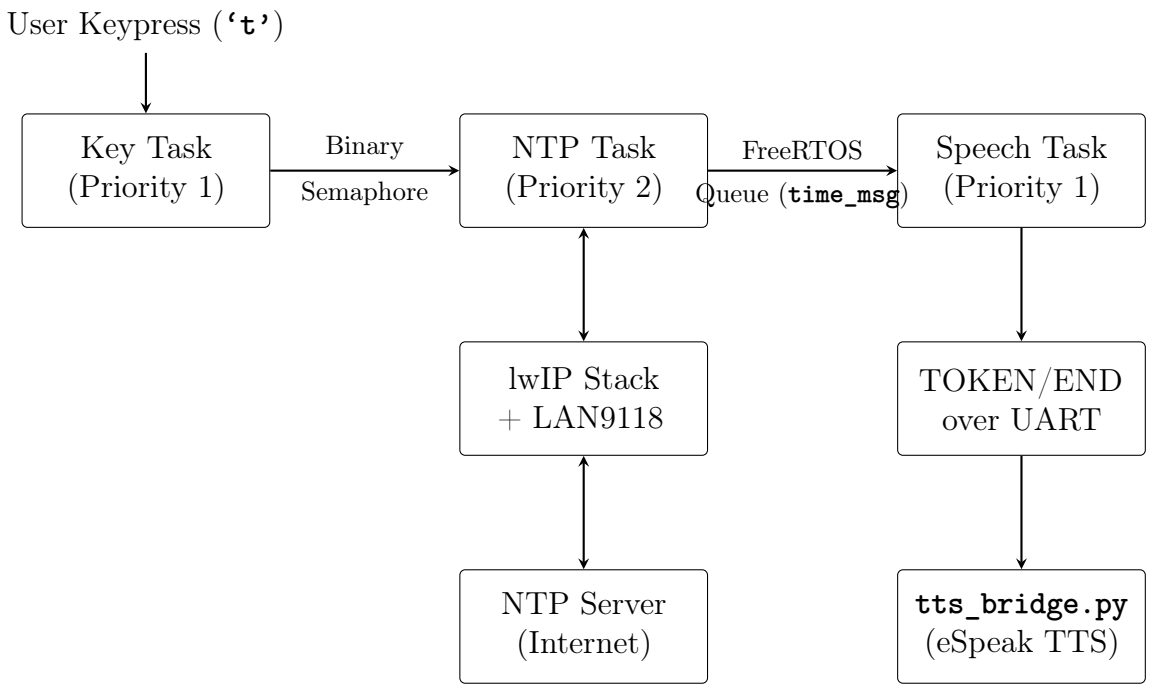

Figure 1.2: System Data Flow Pipeline

1.4 Project Directory Structure

```
|-- Makefile                    # Build system with QEMU launch targets
|-- README.md                   # This file
|-- config/
|   |-- FreeRTOSConfig.h        # RTOS kernel configuration
|   |-- lwipopts.h              # lwIP stack configuration
|   `-- arch/
|       `-- cc.h                # lwIP platform abstraction (types, asserts)
|-- freertos/                   # FreeRTOS kernel source
|   |-- tasks.c, queue.c, list.c, timers.c, event_groups.c
|   `-- portable/
|       |-- GCC/ARM_CM3/port.c  # Cortex-M4 port (SVC, PendSV, SysTick)
|       `-- MemMang/heap_4.c    # Dynamic memory allocator
|-- include/
|   |-- time_msg.h              # IPC message structure
|   `-- speech_tokens.h         # Token constants + word lookup table
|-- linker/
|   `-- stm32.ld                # Linker script
|-- lwip/                       # lwIP 2.2.2 source tree
|   `-- src/
|       |-- core/               # UDP, DNS, DHCP, ARP, ICMP, IP...
|       |-- include/            # lwIP headers
|       `-- netif/              # Ethernet framing
|-- network/
|   |-- lwip_init.c             # Stack init, DHCP start, polling loop
|   |-- ntp_client.c            # NTP protocol implementation (raw UDP)
|   `-- netif_qemu.c            # LAN9118 Ethernet driver for QEMU
|-- scripts/
|   |-- tts_bridge.py           # Python TTS bridge (espeak integration)
```

```
|   |-- run_qemu.sh            # QEMU launch helper
|   `-- setup_network.sh       # TAP/bridge network setup (optional)
|-- src/
|   |-- main.c                 # Entry point, UART driver, IPC setup
|   |-- key_task.c             # UART input monitor task
|   |-- ntp_task.c             # Network + NTP client task
|   `-- speech_task.c          # Token-based speech output task
`-- startup/
    `-- startup_stm32.s        # Vector table, Reset_Handler, init
```

1.5 Memory Map

The linker script (`linker/stm32.ld`) defines the following memory layout:

Region	Address Range	Size	Contents
Flash	`0x00000000` – `0x003FFFFF`	4 MB	Vector table, `.text`, `.rodata`, `.ARM.exidx`
RAM	`0x20000000` – `0x203FFFFF`	4 MB	`.data`, `.bss`, heap (4 KB newlib + 64 KB FreeRTOS + 16 KB lwIP), stack

The startup code (`Reset_Handler`) copies `.data` from Flash to RAM and zeroes `.bss` before calling `main()`.

2 RTOS Design

2.1 FreeRTOS Configuration

The kernel is configured via `config/FreeRTOSConfig.h`:

Parameter	Value	Rationale
`configCPU_CLOCK_HZ`	25 MHz	STM32 system clock
`configTICK_RATE_HZ`	1000 Hz	1 ms tick resolution for lwIP timers
`configTOTAL_HEAP_SIZE`	64 KB	Task stacks + IPC + lwIP memory pools
`configMAX_PRIORITIES`	5	Three levels used (1, 2, 3)
`configUSE_PREEMPTION`	1	Preemptive scheduling enabled
Memory scheme	`heap_4.c`	Coalescing first-fit allocator

FreeRTOS port handlers are mapped to standard CMSIS exception names:

- `SVC_Handler` → `vPortSVCHandler` (initial context switch)
- `PendSV_Handler` → `xPortPendSVHandler` (context switching)
- `SysTick_Handler` → `xPortSysTickHandler` (1 ms tick)

2.2 Task Architecture

Three application tasks form a pipeline. The NTP Task has the highest priority because it must service the lwIP polling loop to keep the network stack responsive.

Task	Function	Priority	Stack	Role
Key Task	`key_task()`	1 (Low)	256 words	Polls UART0 for `'t'` keypress
NTP Task	`ntp_task()`	2 (Medium)	1024 words	Network init, lwIP polling, NTP queries
Speech Task	`speech_task()`	1 (Low)	512 words	Converts time to TOKEN/END stream

2.3 Key Task (`src/key_task.c`)

- Polls the CMSDK UART0 receive register (`UART0_STATE & RXFULL`) every 50 ms using `vTaskDelay()`
- On detecting `'t'` or `'T'`, gives the binary semaphore (`xSemaphoreGive`)
- Runs at Priority 1; yields the CPU between polls

2.4 NTP Task (`src/ntp_task.c`)

- **Phase 0**: Initializes the lwIP network stack (must happen after the FreeRTOS scheduler starts because `sys_now()` uses `xTaskGetTickCount()`)
- **Phase 1**: Polls lwIP while waiting for DHCP to assign an IP address
- **Phase 2**: Main loop — takes the semaphore with a 100 ms timeout; if granted, performs an NTP query with up to 3 retries; always calls `lwip_periodic_handle()` to keep the network alive
- On successful NTP response, sends a `time_msg` struct to the Speech Task via `xQueueSend()`

2.5 Speech Task (`src/speech_task.c`)

- Blocks indefinitely on `xQueueReceive(time_queue, ...)`
- On receiving a `time_msg`, calls `speak_time()` which emits a TOKEN/END sequence:
```
TOKEN TIME
TOKEN IS
TOKEN TWENTY_TWO
TOKEN HOURS
...
END
```
- Number-to-word conversion uses a 60-entry lookup table in `speech_tokens.h`

2.6 Inter-Task Communication (IPC)

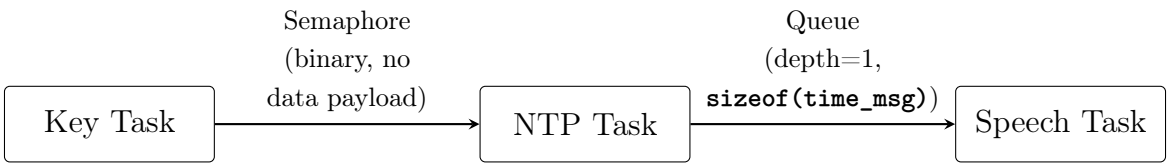

Figure 2.1: Task Inter-Process Communication Map

Both are created in `main()` before the scheduler starts, and accessed via `extern` declarations in each task file.

Primitive	Type	Purpose
`time_request_sem`	Binary Semaphore	Signals "user wants the time" from Key → NTP
`time_queue`	Queue (depth 1)	Carries `time_msg` {hour, minute, second} from NTP → Speech

2.7 Fault Handlers

Custom C fault handlers in `main.c` override the weak defaults from startup assembly. They print diagnostic messages to UART before halting:

- `HardFault_Handler` — catches illegal memory access, undefined instructions
- `MemManage_Handler` — MPU violations
- `BusFault_Handler` — bus errors during memory transactions
- `UsageFault_Handler` — division by zero, unaligned access

3 Networking Implementation

3.1 lwIP Configuration

lwIP operates in **NO_SYS mode** (`NO_SYS=1`), meaning there is no OS abstraction layer. Instead, the application manually polls the stack from the NTP Task. Key settings in `config/lwipopts.h`:

Feature	Setting	Notes
System mode	`NO_SYS=1`	Polling mode, no threads/mutexes
UDP	Enabled	Required for NTP (port 123) and DHCP
TCP	Disabled	Not needed, saves ∼10 KB code
DHCP	Enabled	Automatic IP assignment via QEMU SLIRP
DNS	Enabled	Available but NTP uses hardcoded IP
ARP	Enabled	Required for Ethernet communication
ICMP	Enabled	Allows ping for debugging
IPv6	Disabled	Only IPv4 is used
MEM_SIZE	16 KB	lwIP heap for pbufs and PCBs
PBUF_POOL_SIZE	16 × 1536 B	Receive buffer pool (∼24 KB)
ACD	Disabled	Avoids linker errors in lwIP 2.2.2

3.2 Network Initialization Sequence

The initialization happens inside the NTP Task (post-scheduler) via `network_init()` in `network/lwip_init.c`:

```
1. lwip_init()            // Initialize all lwIP sub-modules
2. netif_add()            // Register the LAN9118 driver with lwIP
3. netif_set_default()    // Set as the default network interface
4. netif_set_up()         // Bring the interface up
5. dhcp_start()           // Begin DHCP discovery
```

A status callback (`netif_status_callback`) prints the assigned IP address when DHCP completes.

3.3 LAN9118 Ethernet Driver (`network/netif_qemu.c`)

The driver interfaces with the **SMSC LAN9118** Ethernet controller emulated by QEMU at base address `0x40200000`.

3.4 Register Access

- **Direct registers**: TX/RX data FIFOs, status FIFOs, HW_CFG, IRQ, etc.
- **Indirect MAC registers**: Accessed through `MAC_CSR_CMD` / `MAC_CSR_DATA` with a busy-wait loop and timeout protection.

3.5 Initialization (`lan9118_hw_init`)

1. Read `ID_REV` register to verify the controller is present (expected: `0x01180001`)
2. Read `BYTE_TEST` register to confirm endianness (`0x87654321`)
3. Perform a soft reset via `HW_CFG |= SRST`, wait for bit to clear
4. Configure TX (`TX_CFG = TX_ON | TXSAO`) and disable interrupts (`INT_EN = 0`)
5. Write MAC address to `MAC_ADDRH` / `MAC_ADDRL` via indirect CSR access
6. Enable MAC TX and RX (`MAC_CR = TXEN | RXEN | FDPX`)

3.6 Transmit (`lan9118_low_level_output`)

1. Write `TX_CMD_A` (buffer size + first/last segment flags)
2. Write `TX_CMD_B` (packet length)
3. Copy pbuf payload to `TX_DATA_FIFO` in 32-bit words
4. Read `TX_STATUS_FIFO` to confirm completion

3.7 Receive (`lan9118_poll`)

1. Check `RX_FIFO_INF` for available data
2. Read `RX_STATUS_FIFO` for packet length and status
3. Read packet data from `RX_DATA_FIFO` into a pbuf chain
4. Pass to lwIP via `netif->input()` → `ethernet_input()`

3.8 QEMU Networking Modes

Mode	Makefile Target	QEMU Flag	Description
User (NAT)	`make flash`	`-nic user`	SLIRP userspace NAT, no host config needed. Guest gets `10.0.2.15`
TAP Bridge	`make flash-tap`	`-nic tap,ifname=tap0`	Host TAP interface, requires `setup_network.sh`. Full network access

3.9 DHCP Flow

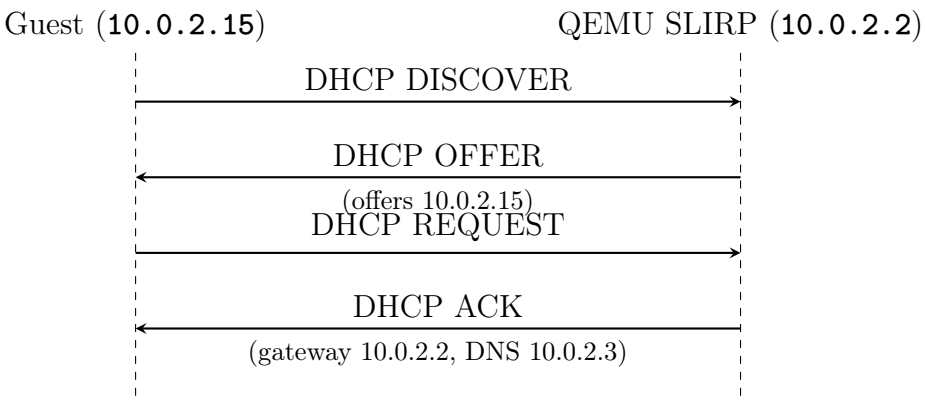

Figure 3.1: DHCP Negotiation Sequence

4 NTP Protocol Explanation

4.1 Protocol Overview

The **Network Time Protocol (NTP)** synchronises clocks over packet-switched networks. This project implements a **SNTP (Simple NTP) client** — it sends a single request and extracts the server's transmit timestamp without performing the full clock discipline algorithm.

- **Transport**: UDP port 123
- **Packet size**: 48 bytes (fixed)
- **Reference**: RFC 5905 (NTPv4), RFC 4330 (SNTP)

4.2 NTP Packet Structure

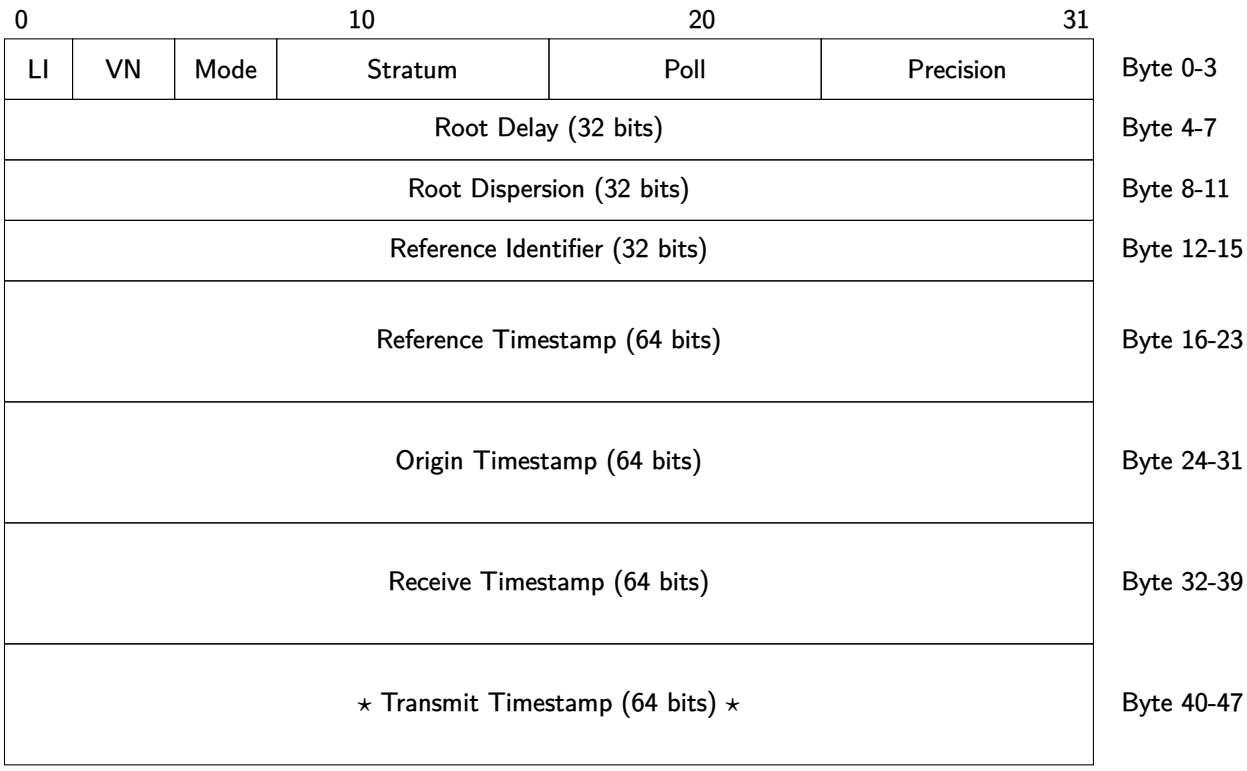

Figure 4.1: NTPv4 Protocol Packet Structure

4.3 Client Request Construction

The client sends a 48-byte packet with only byte 0 set:

```
ntp_request[0] = 0x23;
// Binary: 00 100 011
//         LI=0 (no warning)
//         VN=4 (NTP version 4)
//         Mode=3 (client)
```

All other bytes are zero. This is a valid SNTP client request per RFC 4330.

4.4 Response Parsing

From the server's 48-byte response, we extract the **Transmit Timestamp** at bytes 40–43 (seconds portion only, ignoring the fractional part at bytes 44–47):

```
ntp_timestamp = (buf[40] << 24) | (buf[41] << 16) | (buf[42] << 8) | buf[43];
```

This gives us the number of seconds since **1 January 1900 00:00:00 UTC** (the NTP epoch).

4.5 Time Conversion

NTP Epoch → Unix Epoch

```
#define NTP_UNIX_EPOCH_DIFF  2208988800UL  // 70 years in seconds
unix_time = ntp_timestamp - NTP_UNIX_EPOCH_DIFF;
```

UTC → Indian Standard Time (IST)

```
#define IST_OFFSET_SECONDS  19800  // 5 hours * 3600 + 30 minutes * 60
ist_time = unix_time + IST_OFFSET_SECONDS;
```

Extract Hours, Minutes, Seconds

```
seconds_of_day = ist_time % 86400;        // seconds since midnight
hour   = seconds_of_day / 3600;           // 0-23
minute = (seconds_of_day % 3600) / 60;    // 0-59
second = seconds_of_day % 60;             // 0-59
```

4.6 NTP Server

The client uses **time.cloudflare.com** (`162.159.200.1`) as a hardcoded IP address to avoid DNS resolution complexity. The NTP query uses lwIP's raw UDP API:

1. `udp_new()` — create a UDP PCB
2. `udp_bind(pcb, IP_ADDR_ANY, 0)` — bind to any available port
3. `udp_recv(pcb, callback, NULL)` — register async receive callback
4. `udp_sendto(pcb, pbuf, &server_addr, 123)` — send the 48-byte request
5. Poll `lwip_periodic_handle()` in a loop until the callback sets a flag
6. `udp_remove(pcb)` — clean up

The callback extracts the transmit timestamp and sets `ntp_response_received = 1`.

4.7 Speech Token Protocol

The time is announced using a simple text-based protocol over UART:

```
TOKEN THE
TOKEN TIME
TOKEN IS
TOKEN TWENTY_TWO
TOKEN HOURS
TOKEN TEN
TOKEN MINUTES
TOKEN AND
TOKEN EIGHT
TOKEN SECONDS
END
```

The host-side `tts_bridge.py` script parses these lines and converts them to spoken audio via eSpeak: *"The time is twenty two hours ten minutes and eight seconds"*.

5 System Testing

5.1 Build Verification

```
$ make clean && make
```

Expected output:

```
Build complete!
   text    data     bss     dec     hex filename
  45968     100  115676  161744   277d0 output/speaking_clock.elf
```

The firmware is ∼46 KB of code with ∼113 KB of BSS (FreeRTOS heap + lwIP memory pools).

5.2 Boot Sequence Verification

Running `make flash` should produce this output:

```
==================================================
  Speaking Clock - STM32 / FreeRTOS / QEMU
  Network Time Protocol (NTP) Client
==================================================

Creating IPC primitives...
Creating tasks...
Starting FreeRTOS scheduler...

NTP Task: Initializing network stack...
...
Network: IP address obtained via DHCP
  IP:      10.0.2.15
  Netmask: 255.255.255.0
  Gateway: 10.0.2.2
```

```
========================================
  Network-Synchronised Speaking Clock
  Press 't' to announce the time
========================================

NTP Task: Network is ready!
NTP Task: Waiting for time requests...
```

5.3 NTP Query Test

Press `t` in the QEMU console:

```
Key: Time request received!
NTP Task: Processing time request...
NTP: Sending time request...
NTP: Sending to 162.159.200.1:123
NTP: Request sent, waiting for response...
NTP: Received timestamp = 3985605608 (NTP epoch)
NTP: IST time = 22:10:08
NTP Task: Time = 22:10:08 IST
Speech Task: Announcing 22:10:08

--- Speech Output ---
TOKEN THE
TOKEN TIME
...
END
--- End Speech ---
```

5.4 Verification Checklist

Test	Expected Result	Status
Firmware compiles without errors	Clean build, ∼46 KB text	✓
QEMU boots, banner is displayed	ASCII banner over UART	✓
FreeRTOS scheduler starts	All 3 tasks created and running	✓
LAN9118 driver initialised	ID_REV = `0x01180001`	✓
DHCP obtains IP address	`10.0.2.15` with gateway `10.0.2.2`	✓
NTP query returns valid timestamp	Seconds since 1900, matches current time	✓
IST conversion is correct	UTC + 5:30 matches wall clock	✓
Speech tokens are emitted	TOKEN/END sequence over UART	✓
TTS bridge produces audio	eSpeak speaks the time	✓
Multiple presses work	Each press triggers fresh NTP query	✓
Retry on NTP failure	Up to 3 retries with 1s delay	✓

5.5 GDB Debugging

```
# Terminal 1: Start QEMU in debug mode (halted, waiting for GDB)
make debug
```

```
# Terminal 2: Attach GDB
arm-none-eabi-gdb output/speaking_clock.elf
(gdb) target remote :1234
(gdb) break main
(gdb) continue
```

6 Instructions to Run

6.1 Prerequisites

Install the following packages (Ubuntu):

```
# ARM cross-compiler toolchain
sudo apt install gcc-arm-none-eabi
# QEMU system emulator for ARM
sudo apt install qemu-system-arm
# Text-to-speech engine (for speech output)
sudo apt install espeak
# Python 3 (for TTS bridge)
sudo apt install python3
```

6.2 Build

```
cd speaking_clock
make clean && make
```

6.3 Run (Console Only)

Run the firmware in QEMU with user-mode (NAT) networking:

```
make flash
```

Press `t` to request the time. The TOKEN/END output will appear in the console. Press `Ctrl+A`, then `X` to exit QEMU.

6.4 Run with Speech (TTS Bridge)

Launch QEMU through the TTS bridge for actual spoken audio:

```
make tts
```

This starts QEMU as a subprocess with keyboard forwarding. Press `t` to hear the time spoken aloud via eSpeak. Press `Ctrl+C` to exit.

6.5 Makefile Targets Summary

6.6 Troubleshooting

Target	Command	Description
`make`	Build firmware	Compiles all sources, produces `.elf` and `.bin`
`make flash`	Run in QEMU	User-mode networking (NAT), interactive console
`make tts`	Run with TTS	Launches QEMU via Python bridge with eSpeak
`make flash-pipe`	Pipe to TTS	Pipes QEMU output to TTS bridge (no keyboard)
`make flash-tap`	Run with TAP	Uses host TAP interface (requires `setup_network.sh`)
`make debug`	GDB debug	Starts QEMU halted, listening on `tcp::1234`
`make clean`	Clean build	Removes all build artefacts
`make zip`	Package	Creates submission zip excluding `.git` and `output/`

Problem	Solution
`qemu-system-arm: not found`	Install: `sudo apt install qemu-system-arm`
`arm-none-eabi-gcc: not found`	Install: `sudo apt install gcc-arm-none-eabi`
DHCP timeout / no IP address	Ensure QEMU has `-nic user` flag; check host Internet
NTP response timeout	Check host firewall allows outbound UDP port 123
eSpeak not producing audio	Install: `sudo apt install espeak`; check PulseAudio
Hang at "Initializing lwIP stack"	Ensure `_Min_Heap_Size` in linker script is $\geq$ `0x1000`

Licenses

This project is for educational purposes.

- `FreeRTOS` is licensed under the MIT License.
- `lwIP` is licensed under the BSD License.